

QRecauda: un pipeline QRNG reproducible, preinscrito y honesto sobre sus límites, para la recaudación de peajes y Metro

Jefferson Ramirez

2026-10-07 (borrador v0.1 — resultados de E1 y E3 pendientes)

Índice general

Resumen	2
1. Introducción y motivación	3
1.1 El problema de la recaudación	3
1.2 Lo que este trabajo afirma y lo que no	3
1.3 Contribuciones	4
2. Trabajos relacionados	4
3. Amenazas y alcance	5
3.1 Qué se protege	5
3.2 Ataques que el diseño cubre y los que no	5
3.3 Discrepancias declaradas con el manifiesto fundacional	5
4. Arquitectura	7
4.1 Las cuatro macro-capas	7
4.2 Puertos y adaptadores	7
4.3 Contratos de arquitectura verificados por máquina	7
4.4 El plan como DAG verificable	8
5. Métodos	8
5.1 Fuente de bits	8
5.2 Modelo de ruido	8
5.3 Mitigación de lectura propia: twirling con XOR clásico (F4.01, D-009)	9
5.4 Por qué ZNE y PEC no actúan sobre el ruido de lectura	9
5.5 Extractor de Peres	10
5.6 Hash de Toeplitz y longitud segura por LHL	10
5.7 Estimación de min-entropía: MCV frente a SP 800-90B	11
5.8 Validación estadística: NIST SP 800-22	11
5.9 Cifrado AES-256-GCM y reserva de claves	11
5.10 Métricas de aceptación M1-M7 y aritmética de la cadena	12
6. Diseño experimental preinscrito	13
6.1 Qué significa «preinscribir» aquí	13
6.2 Semillas, controles y desenlaces comunes	13
6.3 Los experimentos	13
P.E1 / E1 — el pipeline entrega claves que cumplen M1-M5 con entrada ruidosa	13
P.E2 / E2 — la mitigación reduce el sesgo de lectura bajo el 1 %	14

P.E3 / E3 — tasa, latencia y caso de uso	14
6.4 Enmiendas fechadas y confesiones de método	15
7. Resultados	16
7.1 Spikes de viabilidad (S.01–S.04)	16
7.2 C.E2 — sesgo antes y después de la mitigación (corrida registrada; veredicto pendiente)	17
7.3 C.E1a–d — calidad de la clave y controles de E1	19
7.4 C.E3 — tasa, latencia y ciclo de cifrado	19
7.5 Resumen de estado de las evidencias	20
8. Discusión	20
9. Limitaciones	21
10. Reproducibilidad	22
10.1 Entorno y versiones	22
10.2 Comandos exactos	22
10.3 Commits relevantes (el historial es el testigo del orden)	22
10.4 Lo que sí y lo que no es reproducible	23
11. Conclusiones	23
Referencias	24
Apéndice A. Mapa de fuentes del repositorio	25
Apéndice B. Decisiones de diseño (resumen)	25

Estado de este documento. Es un informe técnico vivo, escrito mientras las corridas se ejecutan. Toda cifra de resultado remite a una corrida nombrada de registro/corridas/ o a un spike con su ruta; donde la corrida aún no existe, la sección lleva un marcador «**PENDIENTE: corrida X**» y no contiene ninguna cifra ni estimación. Lo marcado (!) sin verificar es hipótesis de trabajo, no premisa. Las cifras «analítico» se derivan a mano de una fórmula escrita al lado y no son resultados. Repositorio: <https://github.com/jramirezgen/qrecauda> (licencia Apache-2.0). Commit de la base de este borrador: fb20457.

Resumen

QRecauda es un pipeline de generación de claves a partir de bits de un circuito cuántico de un solo gate (Hadamard sobre n qubits, medido en la base computacional), pensado para cifrar transacciones de recaudación de peajes y del Metro de Lima. La cadena es: fuente de bits → mitigación del ruido de lectura → extractor de Peres → hash de Toeplitz dimensionado por el Leftover Hash Lemma (LHL) → validación estadística (NIST SP 800-22) y de min-entropía (NIST SP 800-90B) → clave AES-256-GCM. El trabajo se distingue menos por la cadena —clásica— que por cuatro decisiones de método: (i) **honestidad por diseño**: el muestreo de AerSimulator es pseudoaleatorio, por lo que el simulador no aporta entropía cuántica y sólo valida el postprocesamiento; «clave certificada» significa únicamente «supera esta batería», y una batería estadística no certifica origen (un control negativo con un PRNG clásico lo demuestra por diseño); (ii) una **mitigación de lectura propia por twirling** (puerta X aleatoria antes de medir y XOR clásico después) que, a diferencia de ZNE y PEC —que actúan sobre valores esperados y no sobre el error de lectura—, conserva los bits por disparo; (iii) **preinscripción**: umbrales, criterios, controles y desenlaces de cada experimento (E1: calidad de la clave; E2: reducción del sesgo de lectura; E3: tasa y latencia) se fijan en commits

anteriores a la corrida, con enmiendas fechadas, y el juez se niega a emitir veredicto si falta un control; (iv) una **contabilidad de cifras**: ninguna cifra sin corrida nombrada.

A la fecha de este borrador están hechos cuatro spikes de viabilidad (S.01–S.04) y la corrida C.E2 está registrada (48 artefactos, controles C1–C5 en verde), pero **su veredicto aún no se ha emitido**. Una lectura del autor, no el veredicto del juez, aplicando los criterios preinscritos K1–K4 a las celdas registradas: con 400 000 disparos $\times$ 8 qubits, el twirling lleva el sesgo medio por qubit desde 0,0098–0,0104 / 0,0296–0,0304 / 0,0495–0,0505 (niveles bajo, medio y alto de ruido de lectura) hasta 0,00074–0,00131 en las nueve celdas, mientras que ZNE y PEC no mueven el sesgo (cambio relativo $\leq 6,6\%$), tal como la preinscripción declaró. **Las corridas C.E1a–d (calidad de la clave, control negativo y control positivo) y C.E3 (tasa y latencia) están PENDIENTES** y no se reporta ninguna cifra de ellas. Se anticipa, como hipótesis (!) sin verificar, que la métrica de latencia M7 (< 500 ms extremo a extremo) no se cumpla en el perfil que decide; si así fuera, se reportará como resultado negativo. El TRL honesto del conjunto es el de su afirmación más débil —la fuente cuántica en simulador, TRL 3— y sólo subiría con una corrida en hardware IBM real con `job_id`.

Palabras clave: generador cuántico de números aleatorios (QRNG), extracción de aleatoriedad, Leftover Hash Lemma, mitigación de errores de lectura, preinscripción, reproducibilidad, recaudación, Qiskit.

1. Introducción y motivación

1.1 El problema de la recaudación

Los sistemas de cobro automático —telepeaje, tarjetas de transporte, boletaje del Metro— firman, autentican y cifran millones de transacciones pequeñas. Su seguridad descansa en un supuesto poco visible: que las claves y los nonces provienen de una fuente que un atacante no puede predecir. Un generador pseudoaleatorio (PRNG) es una función determinista de su estado; quien reconstruya o robe ese estado reproduce todas las claves futuras (y pasadas, si no hay renovación). La tesis del equipo fundador reza: «La aleatoriedad clásica se predice. La recaudación del Perú no debería.»

El proyecto nació en el Track 4 (criptoseguridad para sistemas de recaudación mediante QRNG) de la Hackatón Qiskit IBM Lima. El contexto de impacto declarado —peajes (OSITRAN, MTC) y el Metro de Lima— motiva el caso de uso: cifrar con AES-256-GCM una transacción de peaje o de Metro con una clave producida por el pipeline. (!) sin verificar: no hay en el repositorio ningún requisito formal de OSITRAN, del MTC o del Metro; los umbrales de tasa y latencia proceden del manifiesto del equipo, no de un requisito del cliente.

1.2 Lo que este trabajo afirma y lo que no

El objetivo es **construir un pipeline QRNG reproducible** y medir, no declarar, siete métricas de aceptación (M1–M7, §5.10). Es un trabajo de ingeniería y de método experimental; no es un resultado de física cuántica ni una certificación:

- **No** afirma entropía cuántica en simulación. El muestreo de AerSimulator usa un generador pseudoaleatorio (spike S.03, §7.1). La demostración con Aer valida el postprocesamiento, no la fuente.
- **No** afirma certificación FIPS/ISO, ni prueba de origen cuántico (que exigiría pruebas de Bell o autoverificación [8]), ni que sus resultados sirvan para claves de producción.
- **No** incluye integración con sistemas de OSITRAN/MTC/Metro, HSM, distribución de claves, ni multiplexado en varios backends (FUNDAMENTO, «Alcance»).
- El hardware IBM real es **opcional** (F3.04/F3.06); ni la demostración ni la versión 0.1.0 dependen de él.

1.3 Contribuciones

1. Una **arquitectura de cuatro macro-capas** (datos, lógica, integración, transversales) con seis contratos de imports verificados por herramienta (§4).
2. Una **mitigación de lectura por twirling** que conserva los bits por disparo, y el análisis de por qué ZNE y PEC no actúan sobre el error de lectura, con la **escala efectiva de ruido** $s = (\lambda + 1)/2$ para ZNE por plegado de la puerta H (§5.3–5.4).
3. Un **protocolo de preinscripción ejecutable**: declaraciones TOML, preinscripciones en prosa, controles obligatorios y un juez que se niega a veredictar sin ellos (§6).
4. Un **registro de discrepancias** con el manifiesto fundacional (nueve, §3.3) y de **enmiendas fechadas**, incluida la confesión de que un piso de calibración se fijó conociendo las semillas de la corrida (§6.4).
5. Resultados **parciales y honestos** (§7), con los pendientes explícitos.

2. Trabajos relacionados

Esta sección es deliberadamente breve y sólo cita fuentes cuyos datos bibliográficos se contrastaron con el registro de DOI (Crossref) o con el repositorio de preprints el 2026-10-07. Donde no pude verificar una referencia no la incluyo.

Generadores cuánticos de números aleatorios. Herrero-Collantes y García-Escartín [1] revisan los QRNG (ópticos y de otro tipo) y su modelado de fuentes y extractores. Un QRNG con circuito de Hadamard sobre qubits físicos es la versión más simple; la certificación del origen exige pruebas de Bell en el estilo de Pironio et al. [8], que este trabajo no hace.

Pruebas estadísticas y estimación de min-entropía. NIST SP 800-22 Rev. 1a [2] define una batería de pruebas estadísticas (monobit, rachas, frecuencia por bloques, ...) y el criterio de proporción de secuencias aprobadas; es una batería que un PRNG bueno pasa, y por eso no puede certificar origen. NIST SP 800-90B [3] especifica la estimación de min-entropía de fuentes de ruido, con el estimador de «valor más común» (MCV) y la batería no-IID, de la que se toma el mínimo de diez estimadores.

Extracción de aleatoriedad. El procedimiento de von Neumann elimina el sesgo de una fuente de bits independientes; Peres [4] lo itera para reciclar la información descartada y acercarse a la entropía de Shannon. Para convertir una fuente con min-entropía conocida en bits casi uniformes se usa un hash universal y el Leftover Hash Lemma de Impagliazzo, Levin y Luby [5] (versión de revista: Håstad et al. [6]); las matrices de Toeplitz son una familia universal eficiente y su uso en hashing (LFSR/Toeplitz) se remonta a Krawczyk [7].

Cifrado autenticado. AES en modo Galois/Counter (GCM) fue presentado por McGrew y Viega [9] y estandarizado en NIST SP 800-38D [10]. La seguridad de GCM exige que el par (clave, nonce) no se repita; QRecauda lo impone con una reserva de claves de uso único (§5.9).

Stack de computación cuántica y mitigación de errores. Qiskit [11] y su simulador Aer proveen SamplerV2, AerSimulator y modelos de ruido. La mitigación de errores sobre valores esperados incluye la extrapolación a ruido cero (ZNE) y la cancelación probabilística de errores (PEC) de Temme, Bravyi y Gambetta [12] (ZNE también en Li y Benjamin [13]); la mitigación de errores de lectura incluye TREX, con twirling de la medición [14], y mthree (matrix-free measurement mitigation) [15]. Una observación que sostiene este trabajo es que ZNE y PEC se definen sobre valores esperados y TREX/M3 corrigen distribuciones o valores esperados, no producen un flujo de bits por disparo: de ahí la mitigación propia (§5.3).

Lo que este trabajo añade. No propone una técnica nueva de extracción ni de mitigación: el twirling de lectura es una variante de la idea de TREX [14] adaptada a conservar bitstrings. Lo que añade es (a) la integración reproducible de punta a punta con límites declarados, (b) un método experimental preinscrito y auditable, y (c) un registro explícito de dónde el método del manifiesto choca con lo que la estadística y las bibliotecas permiten.

3. Amenazas y alcance

3.1 Qué se protege

Que la clave AES-256 de una transacción no sea predecible por un atacante que no ve la fuente. El sesgo de lectura y las correlaciones se tratan con mitigación, Peres y Toeplitz (LHL), y **la longitud de la clave sale de la min-entropía estimada a la entrada del extractor, no de la salida** (la de salida es ≈ 1 por construcción y no informa; §5.7).

3.2 Ataques que el diseño cubre y los que no

ataque	cómo se mitiga	dónde
Sesgo del dispositivo (lectura asimétrica)	mitigación de lectura + extractor; sesgo residual medido (M1)	§5.3, §5.5
Correlación entre bits	estimador SP 800-90B no-IID que decide la longitud segura	§5.7
Reutilización de (clave, nonce) en GCM	ReservaDeClave consume trozos disjuntos; repetir falla	§5.9
Manipulación del texto cifrado	AES-GCM autentica; AutenticacionFallida	§5.9
Agotamiento de entropía	EntropiaInsuficiente: aborta, no degrada	§5.6, §5.9
Secretos en el repositorio	secretos por ruta, nunca por valor; trinquete antisecretos	§4

Fuera de alcance (AMENAZAS): el canal (TLS, red entre peaje, Metro y servidor); la implementación (canales laterales del host, volcados de memoria, dependencias comprometidas); la gestión de claves (no hay KMS ni HSM); el origen cuántico en hardware (sin pruebas de Bell no se certifica); un adversario con acceso al pool de semillas de Toeplitz (D-004, §5.6); y la latencia de cola y red de IBM Quantum, que no entra en M7.

3.3 Discrepancias declaradas con el manifiesto fundacional

El manifiesto del equipo manda sobre el objetivo; su método choca en nueve puntos con lo que las bibliotecas o la estadística permiten. Se declaran antes de actuar (FUNDAMENTO, «Discrepancias declaradas»):

#	El manifiesto dice	Lo que se encontró / decidió	Estado
1	«No dependemos de hardware real» y, en la estrategia, «sobre procesadores físicos de IBM»	Son dos alcances: el backend es un puerto (D-001), simulador por defecto, hardware opcional	decidido

#	El manifiesto dice	Lo que se encontró / decidió	Estado
2	Aer + ruido produce «entropía cuántica»	El muestreo de Aer es un PRNG; Origen.SIMULADOR_AER no reclama origen cuántico (D-002); spike S.03	decidido, demostrado
3	TREX + ZNE + PEC mitigan el bitstream de SamplerV2	ZNE y PEC se definen sobre valores esperados; resilience_level es opción de EstimatorV2 (spike S.02)	verificado en S.02, ver §7.1
4	«TREX vía mthree»	Son técnicas distintas; mthree entrega cuasi- probabilidades, no bits (D-009)	verificado en S.02
5	Min-entropía > 0,9 prueba la calidad	Tras Toeplitz es ≈ 1 por construcción; la informativa es la de entrada	decidido
6	«Claves certificadas»	SP 800-22 no certifica origen ni impredecibilidad; control negativo (D-007)	decidido
7	«TRL 4»	La rúbrica exige $n \geq 3$, protocolo preinscrito y auditable; el TRL real es el de la afirmación más débil (fuente en simulador: TRL 3)	declarado; lo cierra el nodo T.TRL (pendiente)
8	Min-entropía por MCV decide la longitud y la «certifica»	MCV es ciego a la dependencia: una cadena de Markov con permanencia 0,8 (real 0,322 bit/bit, analítico) da MCV 0,9916 y «pasa» (S.04)	decidido
9	M1 < 1 % y M2 > 0,9 como umbrales	A $N = 800\,000$ monobit exige sesgo < 0,0014 (7× más estricto que M1) y M2 sobre 256 bits no puede pasar de 0,785 (analítico, §5.10)	decidido (P.E0)

Una décima salió de la revisión adversarial R.00 (docs/informes/REVISION_DISEÑO_R00.md): medir M1-M5 sólo sobre la salida de Toeplitz haría que cualquier fuente pasase, de modo que las métricas se miden en tres puntos —muestra cruda, mitigada y clave— (hallazgo R.00-1; el revisor, con $p(1) = 0,9$, obtuvo «aprobado» sobre la clave).

4. Arquitectura

4.1 Las cuatro macro-capas

macro-capa	qué es	dónde
Datos	lo que persiste y cruza fronteras: esquemas versionados, informes, serialización canónica, almacén append-only	datos/, puerto Almacen
Lógica	matemática de la entropía, métricas, casos de uso y orquestador	dominio/ (puro), aplicacion/
Integración	todo lo que habla con un sistema externo, uno por puerto, con su SDK confinado	adaptadores/, composicion.py
Transversales	configuración, errores, observabilidad, reproducibilidad, seguridad, concurrencia, empaquetado	transversal/, dominio/errores.py

La presentación y la CLI son bordes de la lógica, no una quinta capa. El flujo es

FuenteDeBits → [Mitigador] → Peres → Toeplitz(LHL) → Validador → Clave → Cifrador
 (prng | aer | ibm) (opcional) dominio dominio (scipy|nistrng) (AES-GCM)

4.2 Puertos y adaptadores

El núcleo no importa ningún SDK: la fuente de bits es el puerto FuenteDeBits (D-001) con tres adaptadores —PRNG clásico, Aer y (reservado) IBM Runtime—; el Mitigador es otro puerto; el Validador y el EstimadorDeEntropía tienen cada uno dos implementaciones independientes que se contrastan (D-006): el validador propio (scipy, fórmulas NIST) frente a nistrng, y la cota MCV del dominio frente al ea_non_iid oficial. Un adaptador cambia sin tocar a otro (ortogonalidad); los SDK pesados son extras opcionales de pyproject.toml.

4.3 Contratos de arquitectura verificados por máquina

Seis contratos de `import-linter(.importlinter)`, ejecutados además desde `tests/arquitectura/test_contratos` (que siembra un `import` prohibido para comprobar que el contrato muerde): C1 capas; C2 adaptadores independientes entre sí; C3 el núcleo y la presentación no importan `transversal/adaptadores/entrada`; C4 el dominio es puro (sin I/O, reloj, aleatoriedad global ni SDK); C5 los SDK sólo en adaptadores; C6 lo transversal es hoja. Se añaden trinquetes por AST (p. ej. ningún `except` silencioso; `os.environ` sólo en configuración; ninguna credencial en el código) y un hook `commit-msg` que rechaza la atribución a modelos. La CI local (`scripts/ci_local.sh: ruff, mypy --strict, lint-imports, comprobaciones del DAG y`

pytest) corre antes de empujar. A 2026-10-07 hay 323 funciones `def test_` (conteo estático; no incluye casos parametrizados).

4.4 El plan como DAG verificable

El plan es un grafo dirigido acíclico de 59 nodos (`plan/`), el estado vive en `registro/nodos.jsonl` (solo se añade) y `ESTADO.md` es generado. Cada eureka tiene un nodo de preinscripción (P.E*), nodos de corrida (C.E*) que descienden de él y un nodo de veredicto. Esto convierte «¿se hizo?» en una pregunta mecánica y evita que un resultado se declare sin su preinscripción. La revisión adversarial independiente de este diseño (R.00) puntuó la arquitectura de capas 6/10 y el DAG hacia TRL 4 con 3,5/10; los hallazgos aceptados entraron al tablero antes que al DAG. La revisión propia previa al release (R.01) está **pendiente**: hasta que corra, el diseño es «10/10 en el papel», no demostrado.

5. Métodos

5.1 Fuente de bits

El circuito es $H^{\otimes n}$ seguido de medición en la base computacional, con $n = 8$ qubits y 400 000 disparos (3,2 M de bits crudos; PE0). La salida se ordena **qubit-mayor**: primero todos los disparos del qubit 0, luego los del 1, ...; la independencia entre qubits y entre disparos es una hipótesis del modelo, no un hecho medido (!) sin verificar; el cross-talk no está modelado). FuenteAer usa `SamplerV2` de Aer; con semilla explícita la salida es una función determinista de la semilla y del circuito (S.03).

Honestidad de origen. Cada Muestra lleva un `Origen`: `PRNG_CLASICO`, `SIMULADOR_AER` o `HARDWARE_IBM`. Sólo la última puede reclamar origen cuántico (`reclama_origen_cuantico`), y no se puede construir sin `job_id`. La transacción cifrada se rotula «validación del pipeline» salvo hardware IBM con `job_id` (D-002).

5.2 Modelo de ruido

Modelo de lectura asimétrica con matriz de confusión por qubit: $p_{10} \equiv p(1 | 0)$ y $p_{01} \equiv p(0 | 1)$, con una convención (`CanalLectura`) en la que $p(\text{leer } 1 | \text{preparado } 0) = p1_dato_0$. Para una superposición uniforme ($H|0\rangle$) la probabilidad de leer 1 es

$$p(1) = \frac{1}{2} + \frac{p(1|0) - p(0|1)}{2}, \quad \text{sesgo analítico} = \left| \frac{p(1|0) - p(0|1)}{2} \right|.$$

Tres niveles **sintéticos** (no medidos en hardware), fijados en `declaraciones/PARAMETROS.toml` antes de medir:

nivel	$(p(1 0), p(0 1))$	sesgo analítico
bajo	(0,01; 0,03)	0,010 (exactamente el umbral M1)
medio	(0,02; 0,08)	0,030
alto	(0,05; 0,15)	0,050

Un cuarto nivel «realista» usa `NoiseModel.from_backend(FakeSherbrooke())` (calibración congelada de `qiskit_ibm_runtime`). Opcionalmente puede añadirse relajación térmica T_1/T_2 sobre la puerta H. **No se modela cross-talk** (Aer no lo trae como canal sencillo y sin acoplamiento medido sería inventar parámetros) y `Pauli-Lindblad/NoiseLearnerV3` no son aplicables en local.

Consecuencia que se aceptó a ciencia cierta: con el nivel bajo de P.E0, el sesgo analítico coincide con el umbral M1 (0,01), de modo que el nivel bajo es un empate con el umbral, no un caso cómodo (§7.2).

5.3 Mitigación de lectura propia: twirling con XOR clásico (F4.01, D-009)

Idea. Por bloque de disparos (aquí 200) y por qubit, una máscara aleatoria $m \in \{0, 1\}$ decide si se aplica una puerta X justo antes de medir; tras medir, el resultado se combina con la misma máscara por XOR clásico. El estado de $H|0\rangle = |+\rangle$ es invariante bajo X , de modo que la distribución ideal no cambia, pero el canal de lectura asimétrico queda **simetrizado**: promediado sobre la máscara, el canal efectivo es un canal binario simétrico con probabilidad de error

$$p_{\text{ef}} = \frac{p(1|0) + p(0|1)}{2},$$

y su sesgo para un bit uniforme es 0 (a primer orden, analítico). Por ejemplo, en el nivel medio el sesgo $|-0,03|$ se elimina y queda ruido simétrico $p_{\text{ef}} = 0,05$ que no sesga el bit. **El canal se simetriza, no se elimina**: el ruido de bits simétrico no baja la entropía del bit uniforme pero sí hace que la fuente se aparte del modelo ideal para estados no uniformes (!) sin verificar: el efecto sobre estados no uniformes).

Por qué no basta mitigar los bits ya medidos. El twirling necesita aplicar X antes de medir, así que mitigar **re-ejecuta** el circuito con las máscaras; los bits de la muestra cruda se descartan y la muestra devuelta se marca mitigada=True. Sólo funciona sobre AerSimulator (no hay forma de re-ejecutar hardware desde ese adaptador).

Propiedad clave: conserva los bits por disparo. Un QRNG consume bits, no distribuciones. mthree 3.0.0 devuelve cuasi-probabilidades de conteos (no tiene método por disparo; no hay forma de obtener bits sin remuestrear con un PRNG), de modo que sólo entra como **cifra de contraste**, nunca como fuente de bits (D-009). Las opciones twirling y dynamical_decoupling de SamplerOptions existen, pero se ejecutan en el servicio de IBM y no pueden verificarse en local (S.02, (!) sin verificar si conservan bits y cuánto mueven el sesgo).

Efecto colateral conocido (declarado antes de correr). El twirling por bloques de 200 deja un sesgo local de signo aleatorio entre bloques. El sesgo global se cancela (lo ven bien M1 y M3), pero las pruebas que miran estructura local (frecuencia por bloques, M5; rachas, M4) pueden detectarlo. Un diagnóstico previo del equipo, sin corrida registrada (!), estimó $\chi^2/N \approx 1,8$ y falla de M5 en 7 de 8 semillas sobre la mitigada, mientras que la clave pasa. Una comprobación de escritorio fuera del registro (analítico) calcula que el número esperado de rachas cae un factor $(1 - 4\delta^2)$ con $\delta = 0,03$. Esto motivó las enmiendas de §6.4: M4 y M5 sobre la muestra mitigada son sólo informativas.

5.4 Por qué ZNE y PEC no actúan sobre el ruido de lectura

ZNE y PEC se definen sobre valores esperados de observables (EstimatorV2), no sobre el flujo de bits de SamplerV2 (S.02: las opciones resilience_level, resilience.measure_mitigation y resilience.zne_mitigation son rechazadas por la validación de SamplerOptions). Aquí se aplican al observable $\langle Z \rangle = 1 - 2p_1$ promediado sobre los qubits (ideal: 0 tras H; positivo = más ceros), y el sesgo de bit M1 es $|\langle Z \rangle|/2$.

Además, ambas técnicas atacan **ruido de puerta**, no de lectura:

- **ZNE** pliega cada H en $H(HH)^k$ (factor $\lambda = 1 + 2k$; H es autoinversa) y extrapola $\langle Z \rangle$ a ruido de puerta cero. El plegado no toca el error de medición, que ocurre una sola vez al

final: $\langle Z \rangle$ es el mismo para todo λ y la extrapolación no tiene nada que extrapolar. En el spike S.02, con sólo ruido de lectura (nivel medio), $|\langle Z \rangle| = 0,0600$ en los tres factores y tras extrapolar.

- **PEC** descompone la inversa del canal de puerta en operaciones base $\{I, X, Y, Z, \text{reset}|0\rangle, \text{reset}|1\rangle\}$ con coeficientes q_k , $\sum q_k = 1$, y coste de muestreo $\gamma = \sum |q_k| > 1$. Con perfil de puerta trivial, $\gamma = 1$ y la inversa es la identidad: no hay nada que invertir. Su perfil de ruido es **conocido**, no aprendido: PNA (propagated noise absorption) y Samplomatic exigen un modelo Pauli-Lindblad aprendido en el servicio de IBM y **no están implementados** (!! sin verificar).

Por eso F4.02 los trata como estimadores de $\langle Z \rangle$, declara `Efecto.SIN_EFECTO_ESPERADO` cuando el ruido presente (sólo lectura) no es el que actúan, y **se prueba, no se asume**: el control C4 de E2 mide si mueven $\langle Z \rangle$ (§6.3). La prueba de unidad de ZNE/PEC con ruido de puerta (relajación T_1/T_2 en la H) vive en `tests/adaptadores/test_zne_pec.py` y comprueba que sí corrigen ahí; esas pruebas no son corridas del registro y no aportan cifras a este informe.

Escala efectiva de ruido de ZNE: $s = (\lambda + 1)/2$. El plegado no escala el ruido como λ para este observable. A primer orden, el ruido que inyecta cada H sólo llega a $\langle Z \rangle$ si tras él quedan un número **par** de H (cada H intercambia $Z \leftrightarrow X$). De las λ puertas, esto ocurre en $(\lambda + 1)/2$ de ellas (para λ impar), así que la variable de extrapolación es la escala efectiva $s = (\lambda + 1)/2$ y el cero ideal está en $s = 0$ ($\lambda = -1$), no en $\lambda = 0$. Extrapolar a $\lambda = 0$ (escala="ingenua") deja la mitad del sesgo (lo comprueba `test_zne_escala_ingenua_deja_la_mitad`). (!) sin verificar fuera de su caso: $s(\lambda)$ está deducida a primer orden en el ruido y contrastada con Aer sólo para este circuito ($H^{\otimes n}$, relajación en la H); no vale para otros circuitos ni para otro ruido sin rederivarla. Los pesos de extrapolación (Richardson o recta de mínimos cuadrados) se calculan sobre las escalas efectivas.

5.5 Extractor de Peres

El procedimiento de von Neumann toma pares no solapados: $01 \rightarrow 0$, $10 \rightarrow 1$, 00 y 11 se descartan; es exacto si los bits son IID de cualquier sesgo, y da salida de rendimiento a lo sumo pq por bit de entrada. Peres [4] lo itera: además de los pares distintos, recicla la secuencia de **paridades** ($x_{2i} \oplus x_{2i+1}$) y la de **valores de los pares iguales**, aplicando recursivamente el mismo procedimiento hasta una profundidad d (aquí $d = 8$). La salida es la concatenación de los tres flujos. La exactitud sigue requiriendo bits IID; sobre una fuente con dependencia (p. ej. el sesgo local que deja el twirling por bloques) la garantía no es de por sí válida (!! sin verificar su efecto cuantitativo; ver §9).

5.6 Hash de Toeplitz y longitud segura por LHL

Dado un pool de $|\text{pool}|$ bits tras Peres y una estimación de min-entropía por bit h , la longitud segura es

$$m = \lfloor n h_{\min} - 2 \log_2 \frac{1}{\varepsilon} \rfloor, \quad \varepsilon = 2^{-64},$$

es decir, el LHL [5,6]: al aplicar un hash universal a una fuente de n bits con min-entropía $\geq n h_{\min}$, los m bits de salida están a distancia estadística $\leq \varepsilon$ de uniformes. Con $\varepsilon = 2^{-64}$ la pérdida fija es $2 \cdot 64 = 128$ bits. $\varepsilon = 2^{-64}$ **es un parámetro de diseño, no una garantía** (discrepancia 8). Si $m < 1$ la función aborta con `EntropiaInsuficiente`: no sale ni un bit seguro y no se degrada en silencio.

El hash es $T \cdot x \bmod 2$ con T una matriz de Toeplitz $m \times n$ definida por una semilla de $n + m - 1$ bits uniformes e independientes, calculada con `scipy.linalg.matmul_toeplitz` (vía FFT en float64, redondeo con `numpy rint` y módulo 2). La exactitud del redondeo a n grande no

se asume: una prueba (test_toeplitz_por_fft_es_exacto_a_n_grande) comprueba que $n \geq 2 \cdot 10^5$ no cambia ni un bit, y otra que coincide con la matriz densa módulo 2 en tamaños pequeños; y la linealidad sobre GF(2).

La semilla sale del mismo pool (D-004). No hay segunda fuente en la hackatón, así que datos y semilla son segmentos disjuntos del pool: $n(2 + h) \leq |\text{pool}|$, con n bits de datos y $n + m - 1 \leq n(1 + h)$ de semilla. La independencia entre ambos segmentos es una **hipótesis del modelo de ruido** (!) sin verificar). Un puerto FuenteDeSemilla separado la sustituiría. Un adversario con acceso al pool queda fuera de alcance.

5.7 Estimación de min-entropía: MCV frente a SP 800-90B

MCV (dominio). $H_{\min} = -\log_2 p_u$, con $p_u = \min(1, \hat{p} + z\sqrt{\hat{p}(1-\hat{p})/(n-1)})$, $\hat{p}$ la frecuencia del símbolo más común y $z = 2,5758$ (cuantil 0,995; cota superior al 99 %, NIST 800-90B §6.3.1 [3]). Se coloca en dominio/entropia.py y se contrasta con la implementación de NIST: **coincide a 6 decimales en las tres señales del spike S.04.**

El MCV es ciego a la dependencia. Mira sólo la frecuencia marginal. En el spike S.04, una cadena de Markov con permanencia 0,8 (entropía real 0,322 bit/bit, analítico) da MCV 0,9916 y «pasa»; una secuencia periódica 00001111 da MCV 0,9963 aunque su entropía real es 0 (S.04, spikes/S04_90b/RESULTADO.md).

SP 800-90B (ea_non_iid). Se usa el binario oficial v1.1.8 (usnistgov/SP800-90B_EntropyAssessment), compilado fuera del repositorio con spikes/S04_90b/build_nist.sh y llamado como subproceso con -i -a y 1 bit por símbolo. Exige $\geq 10^6$ muestras. Devuelve el **mínimo** de diez estimadores, que es una cota inferior conservadora: en el spike, para una fuente IID con $p(1) = 0,7$ da 0,322 frente a 0,515 teórico (analítico), y para la Markov 0,170 frente a 0,322. Un umbral contra este estimador debe asumir ese sesgo a la baja; por eso el criterio de rechazo del control positivo de E1 es una **separación** (§6.4), no un valor absoluto de 0,9. Hoy la $h_{\min}$ de entrada que dimensiona la clave sigue siendo la del MCV (estimador="mcv" en los informes); adoptar el 90B para dimensionar es una decisión de F5.02 aún no tomada, con el precio de reducir el rendimiento aun con una fuente perfecta (P.E1, enmienda 2: ≈ 14 % más corta con el 90B medio de la ideal, analítico).

5.8 Validación estadística: NIST SP 800-22

Se implementa con dos validadores **independientes** que un contract test exige que coincidan (D-006): uno propio (scipy, fórmulas NIST) y nistrng 1.2.3. Pruebas usadas: **monobit (M3)**, **rachas/runs (M4)** y **frecuencia por bloques (M5)**. Se verificaron contra los ejemplos publicados de NIST (test_vectores_nist.py) y entre sí en 100 secuencias. Comportamientos de nistrng que se respetan sin forzarlos: Runstest._execute no comprueba el prerequisite de frecuencia (si no es elegible, el p es 0,0 por convención); FrequencyWithinBlockTest fija $M = 20$ y, si $N \geq 100$, usa $N = 99$ y $M = \lfloor n/99 \rfloor$ (descarta la cola); la M5 de este adaptador es esa prueba de bloques, **no** el χ^2 de bytes del validador propio. Una sola p no es el criterio de SP 800-22: lo es la proporción de aprobados sobre N secuencias, $1 - \alpha \pm 3\sqrt{\alpha(1-\alpha)/N}$ [2].

5.9 Cifrado AES-256-GCM y reserva de claves

CifradorAesGcm envuelve cryptography.AESGCM: clave de 256 bits, nonce de 96 bits, y **recuerda** los pares (clave, nonce) usados para cifrar, rechazando repetirlos con NonceRepetido. ReservaDeClave reparte la clave certificada en trozos consecutivos de 352 bits (256 + 96) que salen **una sola vez**; agotada, EntropiaInsuficiente. Una manipulación del texto o del dato asociado (la estación viaja autenticada) produce AutenticacionFallida. El cifrador se contrasta con el vector oficial GCM de 256 bits (test_vector_oficial_gcm_caso_16_de_256_bits).

ServicioDeTransacciones **se niega a cifrar** si la clave no pasa $M1-M5$ (calidad_de_clave_aprobada). Las métricas de la cadena (M6/M7) las juzga E3 de extremo a extremo, no quien cifra (commit 8984147, §6.4). La Transaccion (estación, tarifa en céntimos, tarjeta pseudónima) es sintética: **ningún dato personal real**.

5.10 Métricas de aceptación M1-M7 y aritmética de la cadena

Los umbrales viven en un solo sitio (dominio/metricas.py, D-005), con **desigualdad estricta** (el manifiesto escribe «<» y «>»):

métrica	definición	umbral
M1	sesgo $ \hat{p}(1) - \frac{1}{2} $	$< 0,01$
M2	min-entropía de la clave (cota MCV al 99 %)	$> 0,9$ bit/bit
M3	NIST monobit, valor p	$> 0,01$
M4	NIST rachas, valor p	$> 0,01$
M5	NIST frecuencia por bloques, valor p	$> 0,01$
M6	tasa: bits de clave por segundo	$> 10\,000$ bit/s
M7	latencia extremo a extremo	< 500 ms

El veredicto es **conjuntivo**, y un veredicto vacío no aprueba. M1-M5 se miden en tres puntos (cruda, mitigada, clave) para que Toeplitz no oculte una fuente defectuosa.

Aritmética de P.E0 (fijada antes de medir; se reproduce con una línea de Python):

qué	cuenta	resultado
bits crudos	8 qubits $\times$ 400 000 disparos	3 200 000
rendimiento clave/crudo	medido en la bala trazadora (Peres + Toeplitz, $h \approx 0,995$); no es una corrida del registro	$\approx 0,2988$
clave	$3,2\text{ M} \times 0,2988$	$\approx 956\,000$ bits
M2 por bloques	la cota MCV al 99 % exige $n \geq 1288$ para superar 0,9	bloques de 4096 (esperado $\approx 0,943$)
sesgo máximo que pasa monobit	$2,5758 \cdot 0,5/\sqrt{N}$, $N = 800\,000$	0,00144 (analítico)
90B	mínimo de la herramienta	10^6 bits crudos (se cumple con 3,2 M)
secuencias NIST	956 000/10 240	93; proporción mínima $0,99 - 3\sqrt{0,0099/93} \approx$ 0,958

Cómo se comprueba la cota de M2: con $\hat{p} = 0,5$ y $n = 256$, $p_u = 0,5 + 2,5758 \cdot 0,5/\sqrt{255} \approx 0,581$, o $H_{\min} \approx 0,785$: **M2 sobre una clave de 256 bits no puede pasar de 0,785**; con $n = 1288$ el mismo cálculo da 0,90. Por eso M2 se mide sobre bloques de 4096 bits (mediana, mínimo aparte). Reglas de P.E0: si falta muestra para una prueba, se **suben los disparos** y los umbrales no se relajan; **M1 sobre la clave es informativo, no decisivo** (monobit es más estricto a esta escala); cada eureka se corre con las **tres semillas** declaradas (20261007, 20261008, 20261009) y el veredicto es sobre las tres: un fallo no se repite con otra semilla (selección de semilla prohibida).

6. Diseño experimental preinscrito

6.1 Qué significa «preinscribir» aquí

La regla 5 del FUNDAMENTO obliga a fijar umbral y criterio de cada eureka en docs/preinscripciones/ antes de la corrida. Se ejecuta así: (a) la preinscripción (prosa) y su declaración (declaraciones/*.toml) entran en un commit; (b) la corrida registra en su manifiesto el preinscripcion_sha y el commit del código; (c) qrecauda juzgar se niega si ese commit no precede a la corrida, si el documento cambió después o si falta o falla un control (error CorridaInvalida, código de salida 9); (d) un control fallido no es un «no cumple», es «no se midió bien»: no hay veredicto, hay incidencia; (e) un resultado negativo cierra la eureka como negativo, y reintentar exige una preinscripción nueva (el veredicto anterior se conserva: el registro sólo se añade). Un error de la preinscripción antes de correr se corrige por **enmienda fechada**, en commit propio y anterior a la primera corrida; después de correr no se toca.

6.2 Semillas, controles y desenlaces comunes

Semillas: 20261007, 20261008, 20261009. Cada eureka tiene **controles positivos y negativos** sin los cuales no hay veredicto. Los desenlaces posibles de E1 son CUMPLE, NO CUMPLE, NULO, CUMPLE PARCIAL, e INVÁLIDA (que no es un desenlace sino CorridaInvalida). Se prohíbe relajar un umbral, repetir con otra semilla o mover una métrica a «informativa» después de verla.

6.3 Los experimentos

P.E1 / E1 — el pipeline entrega claves que cumplen M1-M5 con entrada ruidosa

Afirma: con entrada ruidosa (Aer, nivel **medio**, (0,02; 0,08), sesgo analítico 0,030), mitigación → Peres → Toeplitz → validación entrega una clave que cumple M1-M5 y la medición puede distinguir una fuente mala de una buena. No afirma: nada sobre origen cuántico, hardware real, cross-talk ni M6/M7. Cuatro corridas por semilla:

corrida	qué es	criterio (decide)
C.E1a	control negativo : PRNG clásico sin sesgo, sin mitigación	la cruda pasa M1, M3, M4, M5 y la clave pasa M1-M5. Que pase es lo esperado y se declara : M1-M5 no prueban origen cuántico (D-007)
C.E1b	Aer ruidoso sin mitigar	B1: la cruda falla M1 o M3. D1 (control): $ \text{sesgo cruda} - 0,030 \leq 0,002$. Se reporta, sin decidir: M4/M5, el 90B y si la clave pasa
C.E1c	Aer ruidoso con twirling (bloque 200)	M-1: la mitigada pasa M1 y M3 ; M-2: la clave pasa M1-M5 . Informativos: M4/M5 y 90B de la mitigada, proporción NIST de la clave
C.E1d	control positivo con fuentes sintéticas de 10^6 bits	ver abajo

C.E1d, una fuente por semilla: sesgada ($p(1) = 0,7$): M1 y M3 fallan; periódica 00001111: falla alguna de M1/M3/M4/M5 o 90B < techo; Markov (permanencia 0,8): **90B < techo y MCV** $\geq 0,9$ (el MCV la deja pasar); ideal (PRNG): pasa M1, M3, M4, M5 y 90B $\geq$ piso. Controles de E1: **N1** (la batería acepta una fuente ideal), **D1** (el detector ve el sesgo inyectado) y **P1** (C.E1d). Tasa de falsa alarma de N1 (analítico, supone independencia (!)): $1 - 0,99^{18} \approx 0,165$ por 18 pruebas a $\alpha = 0,01$; se acepta a ciencia cierta y, si ocurre, la corrida es inválida y **no se repite con otra semilla**.

Discrepancia con el plan, declarada. El nodo C.E1d pedía que la Markov se rechazase «por $h_{\min}$ y no por NIST». La preinscripción no lo adopta: la Markov cambia de valor en el 20 % de los pasos frente al 50 % esperado, y las rachas (M4) la ven de lejos (comprobación de escritorio fuera del registro: monobit $p \approx 0,009$, rachas $p \approx 0$, frecuencia por bloques $p \approx 10^{-36}$, (!) no es resultado). Lo que se preinscribe es lo que sostiene la discrepancia 8: la ciega el MCV, no NIST.

P.E2 / E2 — la mitigación reduce el sesgo de lectura bajo el 1 %

Afirma: con ruido de lectura asimétrico inyectado en Aer, la mitigación de lectura baja el sesgo del flujo de bits por debajo de M1. No afirma: origen, correlaciones, hardware ni cross-talk. Se declaró antes de medir qué técnica puede mover qué ruido: «**sin efecto esperado**» de **ZNE/PEC sobre lectura es un veredicto válido**, no un fallo de E2.

Estadístico. Sesgo de una celda = media sobre los 8 qubits de $|\hat{p}_q - \frac{1}{2}|$, con $\hat{p}_q$ la frecuencia de unos del qubit. Tiene sesgo al alza a ruido cero: su **piso analítico** a 400 000 disparos es $E|N(0, 0,5/\sqrt{400\,000})| \approx 0,0006$. **Intervalo:** bootstrap paramétrico por qubit (unos $_q^* \sim \text{Binomial}(\text{shots}, \hat{p}_q)$), $B = 10\,000$, percentiles 2,5 y 97,5 (!) supone disparos independientes; el twirling por bloques puede añadir varianza que este IC ignora).

Criterios del twirling (conjuntivos, por celda nivel $\times$ semilla; 9 celdas decisivas):

id	criterio	valor
K1	residuo < 0,01 (M1, estricto)	0,01
K2	límite superior del IC95 del residuo < 0,01	0,01
K3	residuo $\leq 0,003$ (3,3 $\times$ bajo M1; $\approx 5\times$ el piso 0,0006)	0,003
K4	crudo/residuo ≥ 5 , sólo si crudo $\geq 0,01$	5

El nivel realista (FakeSherbrooke) se juzga con las mismas K1-K4 pero con **veredicto propio, fuera de la conjunción** de los tres sintéticos. Un residuo cuyo IC95 contenga 0,01 es INCONCLUSO y cabe una ampliación de disparos (800 000 y, si persiste, 1 600 000, umbrales intactos). Un factor < 1,5 con crudo $\geq 0,01$ es un **NULO** que se publica.

Controles de E2. C1 (positivo del detector): $|\text{crudo} - \text{analítico}| \leq 0,002$ en medio y alto; C2 (negativo): twirling sin NoiseModel, residuo $\leq 0,003$; C3 (negativo): twirling con canal simétrico (0,05; 0,05), residuo $\leq 0,003$; C4 (puerta de ZNE/PEC): «sin efecto» si $|\langle Z \rangle_{\text{después}} - \langle Z \rangle_{\text{antes}}| < 0,1 |\langle Z \rangle_{\text{antes}}|$ —si mueven $\geq 10\%$, es un **hallazgo a reportar**, no un error, y no se reinterpreta hacia lo esperado—; C5: mthree como contraste (no decide; si no corre, «no medido»).

P.E3 / E3 — tasa, latencia y caso de uso

Afirma: el pipeline completo, en una PC con AerSimulator, entrega **M6 > 10 000 bit/s** y **M7 < 500 ms**, con una transacción cifrada con la clave que produjo. No afirma: nada sobre IBM real (M7 **no** incluye cola ni red), ni que 500 ms o 10 kbit/s sean lo que exijan OSITRAN, el MTC o el Metro (!) sin requisito en el repositorio).

Definiciones, fijadas antes de medir. t_{rep} es el tiempo extremo a extremo de una repetición: fuente → mitigación → extracción (Peres + Toeplitz) → validación (M1-M5 en cruda/mitigada/clave y el 90B) → una transacción cifrada y descifrada. **M6** = $\sum \text{len}(\text{clave}) / \sum t_{\text{rep}}$ por semilla; **M7** = percentil 95 de t_{rep} sobre $N = 30$ repeticiones tras 3 de calentamiento, con `numpy.percentile(..., 95, method="higher")` (el 29.º de 30, conservador). Reloj monotónico `time.perf_counter_ns`; **un hilo** (`threadpoolctl`, `OMP_NUM_THREADS=1`, `max_parallel_threads=1` en Aer); candado de máquina; una sola máquina; carga previa $< 1,0$ (!) umbral arbitrario fijado hoy). Validez de «un hilo»: $\text{CPU/pared} \leq 1,10$ en cada repetición, si no la corrida es inválida.

Perfiles. El **perfil A** (lote completo de 3,2 M de bits, una repetición = una generación entera de clave + una transacción) decide. El **perfil B** (reserva ya generada; ciclo siguiente+cifrar+descifrar sobre 1 000 transacciones, 500 peaje y 500 Metro) no decide y no puede rescatar un A fallido: existe para decir con honestidad «el cuello es el lote, no la cifra». Controles **U1-U5**: ida y vuelta 1 000/1 000; alteración de 1 bit del cifrado o del dato asociado detectada en 100 %; clave de otra transacción falla en 100 %; ningún (clave, nonce) repetido; todos los rótulos «validación del pipeline».

Expectativa declarada como hipótesis, no como resultado. PE3 anticipa que A es el candidato a **no** cumplir M7 (3,2 M de disparos con `memory=True`, un bucle Python por disparo en el twirling, más NIST y 90B sobre $\geq 10^6$ bits). Una estimación informal del equipo, sin corrida registrada, sitúa una repetición del perfil A en el orden de unos 6 s frente a los 500 ms del umbral (!) sin verificar hasta C.E3; **no es un resultado y no se usa como tal**. Si M7 falla, es un veredicto válido; **no se reduce el lote ni se cambia el perfil después de medir**, y optimizar es un nodo nuevo con preinscripción nueva (E3b).

6.4 Enmiendas fechadas y confesiones de método

Todas anteriores a la corrida que afectan y sin cambio de ningún umbral de M1-M7. Se listan porque cada una es una oportunidad de sesgo del investigador y el lector debe poder juzgarlas.

fecha	documento	enmienda	commit	por qué y qué NO cambia
2026-10-07	E3	El guard de cifrado de ServicioDeTransacciones juzga sólo M1-M5; M6/M7 las juzga C.E3 de extremo a extremo (salida A)	8984147, 09da88c, 5a72939	Las M6/M7 internas se miden hasta Toeplitz y podrían impedir cifrar; no se salta el guard ni se relaja un umbral
2026-10-07	E3	M1-M5 en el punto «mitigada» son sólo informativas en E3	5a72939	Misma causa que la de E1; T1-T4 no leen M1-M5
2026-10-07	E1	M5 sobre la mitigada es informativa (decisión previa, delegada)	dae22a6	Diagnóstico del equipo sin corrida registrada (!)

fecha	documento	enmienda	commit	por qué y qué NO cambia
2026-10-07	E1 (1)	M4 sobre la mitigada pasa a informativa; M-1 decide por M1 y M3	1a06602	Misma causa; basada en una simulación de escritorio, no en una corrida registrada (!) . La clave sigue decidiendo M1-M5
2026-10-07	E1 (2)	P1 pasa de «ideal con 90B > 0,9» a separación del instrumento: piso 0,8 (ideal) y techo 0,5 (Markov y periódica)	37b89ca, 447c907	Ver abajo

La confesión del piso de P1. El 90B de la fuente ideal se midió antes de correr E1, pero **sobre las semillas declaradas** (20261007/8/9) y ocho más (20261010-20261017): spikes/S04_90b/calibracion_pl.py → spikes/S04_90b/calibracion_pl.json, 1 000 000 de bits por fuente y semilla. Resultado: la ideal da 90B de **0,821 a 0,903** (sólo 2 de 11 superan 0,9), la Markov 0,1699-0,1717, la sesgada 0,3208-0,3237, la periódica 0. El criterio original (ideal > 0,9) fallaba en 9 de 11 fuentes ideales: el instrumento resultaba inválido por una propiedad suya (el mínimo de 10 estimadores es una cota inferior conservadora, incluso con $h = 1$) y no por la fuente; y el 0,9 de M2 es el de la clave de salida tras Toeplitz, otra magnitud. El piso 0,8 es el mínimo observado (0,821) redondeado hacia abajo a 0,1; el techo 0,5 deja un hueco de 0,3 con el piso. **Quien fijó el piso conocía los tres valores de las semillas declaradas (0,821; 0,902; 0,832).** Por tanto el control P1 pasará en ellas salvo cambio de código, no es una medida «ciega», y para una semilla futura la cola por debajo de 0,8 no se ha medido (!! sin verificar). La pregunta que P1 responde pasa a ser una separación («¿el estimador distingue lo bueno de lo malo?»), no «¿la ideal alcanza 0,9?».

7. Resultados

Regla de esta sección: cada cifra va con su corrida o spike. Lo no corrido lleva el marcador **PENDIENTE**.

7.1 Spikes de viabilidad (S.01-S.04)

Los spikes son sondas de viabilidad con run.py → resultado.json y RESULTADO.md (spikes/S0*/); no son eureka ni llevan preinscripción, pero sostienen decisiones.

S.01 — Versiones (spikes/S01_versiones/). Conviven en un mismo entorno Python 3.13.14: qiskit 2.5.2, qiskit-aer 0.17.2, qiskit-ibm-runtime 0.50.0, mthree 3.0.0 (importa y trae M3Mitigation), nistrng 1.2.3 y cryptography 50.0.2. Aer + SamplerV2 + get_bitstrings()

corre un $H^{\otimes 8}$ de 1000 disparos. Falta qiskit-ibm-transpiler (F3.03), por lo que se usa el pass manager local y se registra.

S.02 — Alcance de la mitigación (spikes/S02_mitigacion_alcance/). Ruido de lectura (0,02; 0,08) (sesgo analítico 0,0300), 100 000 disparos, semillas 11/22/33:

técnica	bits por disparo	local (Aer)	observable	sesgo medio por qubit
SamplerV2.options (twirling, DD) mthree 3.0.0	¡(!) sin verificar no	no sí	bitstrings cuasi-probabilidades bitstrings	(!) sin verificar 0,0299 → 0,0016
Twirling propio (X + XOR)	sí	sí		0,0299 → 0,0020
ZNE	no	no	valores esperados	$ \langle Z \rangle $: 0,0600 → 0,0600
PEC	no	no (!) sin verificar	valores esperados	no ejecutado (!)

SamplerOptions (v0.50.0) rechaza resilience_level y resilience con ValidationError. El residuo del twirling por semilla fue 0,00172/0,00141/0,00287, a la altura del ruido estadístico (~0,0025 con 100 000 disparos); el de mthree 0,00191/0,00162/0,00144. mthree entrega cuasi-probabilidades sin probabilidades negativas; no hay bitstrings sin remuestrear con un PRNG.

S.03 — Aer es pseudoaleatorio (spikes/S03_aer_pseudoaleatorio/, $H^{\otimes 8}$, 2048 disparos): con la misma semilla (42) dos corridas dan bitstrings idénticos disparo a disparo, tanto en AerSimulator.run como en SamplerV2; con semillas 42 y 43 las secuencias difieren; con NoiseModel + ReadoutError y semilla fija sigue siendo determinista. SamplerV2(seed=None) pasa seed_simulator=None a backend.run; el binario C++ de Aer importa std::random_device. (!) sin verificar: que la semilla por defecto salga de std::random_device y no de otro mecanismo (no se leyó el fuente C++). Lo que **sí** puede afirmar la demostración: valida el pipeline y el modelo de ruido es reproducible con semilla. Lo que **no**: entropía cuántica, ni claves reales con semilla fija; que la salida sin semilla «parezca» impredecible no la hace cuántica.

S.04 — Estimador 90B (spikes/S04_90b/), 10^6 bits, semilla 20261007, -i -a, $\approx 0,5$ s por señal:

señal	h esperada	NIST (mínimo)	estimador del mínimo	MCV (NIST = propio)
IID $p(1) = 0,7$	0,515	0,322	Compression	0,5123
Markov, permanencia 0,8	0,322	0,170	Collision	0,9916
Periódica 00001111	0	0	Collision	0,9963

Para la Markov, por estimador: MCV 0,992; Collision 0,170; Markov 0,325; Compression 0,213; t-Tuple 0,331; LRS 0,557; MultiMCW 0,486; Lag 0,319; MultiMMC 0,319; LZ78Y 0,319. El mínimo exigido por la herramienta es 10^6 muestras. La compilación depende de red la primera vez y de -march=native (binario no portable entre CPUs).

7.2 C.E2 — sesgo antes y después de la mitigación (corrida registrada; veredicto pendiente)

Estado de la evidencia. La corrida C.E2 existe como manifiesto (registro/corridas/C.E2.json) con **48 artefactos** (registro/corridas/C.E2_<semilla>_e2_NNN.json, 16 por semilla), el

preinscripcion_sha 43e26492... (el commit de P.E2), el commit del código 26e5b41d..., y los controles del manifiesto **C1 = C2 = C3 = C4 = C5 = verdadero**. Entorno del manifiesto: Python 3.13.14, qiskit 2.5.2, qiskit-aer 0.17.2, qiskit-ibm-runtime 0.50.0, mthree 3.0.0, nistrng 1.2.3, numpy 2.5.3, scipy 1.18.1, Linux/WSL2. **El veredicto de E2 aún no se ha emitido** (qrecauda juzgar E2 no se ha corrido; no hay entrada en registro/ ni ESTADO.md lo marca como hecho). Lo que sigue es una **lectura del autor aplicando los criterios K1-K4 de la preinscripción a las celdas registradas**; no sustituye al juez y, si el juez discrepa, manda el juez.

Tabla principal (400 000 disparos $\times$ 8 qubits por celda; “crudo” es la celda ninguna de la misma semilla y nivel; residuo = sesgo medio por qubit con twirling propio, bloque 200; IC sup. = límite superior del IC95; “máx/qubit” = máximo por qubit del residuo; factor = crudo / residuo). Fuente: registro/corridas/C.E2_<semilla>_e2_001/003/005/007 (twirling) y ..._000/002/004/006 (crudo).

nivel	semilla	crudo	residuo	IC95 sup.	máx/qubit	factor	K1 K2 K3 K4
bajo	20261007	0,00978	0,00084	0,00149	0,00162	11,6	sí n/a (crudo < 0,01)
bajo	20261008	0,01039	0,00074	0,00143	0,00255	13,9	sí sí
bajo	20261009	0,00990	0,00089	0,00151	0,00167	11,1	sí n/a (crudo < 0,01)
medio	20261007	0,02958	0,00092	0,00159	0,00292	32,3	sí sí
medio	20261008	0,03036	0,00086	0,00153	0,00214	35,2	sí sí
medio	20261009	0,02997	0,00098	0,00168	0,00238	30,5	sí sí
alto	20261007	0,04953	0,00108	0,00174	0,00336	45,9	sí sí
alto	20261008	0,05046	0,00103	0,00163	0,00146	49,1	sí sí
alto	20261009	0,04967	0,00131	0,00194	0,00333	37,8	sí sí
realista	20261007	0,00046	0,00075	0,00143	0,00142	0,6	sí n/a
realista	20261008	0,00099	0,00061	0,00131	0,00161	1,6	sí n/a
realista	20261009	0,00084	0,00036	0,00113	0,00086	2,3	sí n/a

Lectura (autor, no veredicto).

1. **Niveles sintéticos.** En las nueve celdas el residuo (0,00074-0,00131) es menor que el umbral M1 (0,01), su límite superior de IC95 ($\leq 0,00194$) también, y está bajo el residuo máximo K3 (0,003). K4 (factor ≥ 5) se cumple donde aplica: 30,5-35,2 en medio, 37,8-49,1 en alto, y 13,9 en bajo (semilla 20261008). El residuo observado es del orden de 1,2-2,2 veces el piso analítico de 0,0006: el twirling lleva el sesgo a la altura del ruido de muestreo.
2. **El nivel bajo es un empate con el umbral, como se declaró.** El crudo del nivel bajo es 0,00978, 0,01039 y 0,00990: **dos de las tres semillas ya cumplen M1 sin mitigar**, de modo que K4 «no aplica» en ellas y el nivel bajo es poco informativo sobre la mitigación.
3. **Nivel realista (FakeSherbrooke).** El crudo es 0,00046-0,00099, ya al nivel del piso: el modelo de ruido de lectura del backend falso casi no tiene asimetría. El twirling no mejora (factor 0,6-2,3) porque no hay sesgo que reducir; el residuo (0,00036-0,00075) no se distingue del piso. Es consistente con «cumple», pero **no es evidencia de que la mitigación funcione en el realista**: el realista no la pone a prueba. (!) sin verificar: que from_backend aplique en el circuito del twirling el error de puerta de la H (no es nativa del backend falso; si no hay transpilación su error no entra).
4. **Máximo por qubit.** En las celdas sintéticas con twirling llega a 0,00336 (alto, 20261007) y 0,00333 (alto, 20261009), por encima de 0,003; K3 juzga la media por qubit, no el máximo, y la preinscripción sólo pide reportar el máximo en el realista. Se señala porque un qubit concreto puede quedar peor que la media.
5. **Controles de instrumento.** C2 (twirling sin ruido): residuo 0,00078-0,00089 (IC sup. $\leq 0,00153$); C3 (canal simétrico (0,05; 0,05)): 0,00066-0,00092 (IC sup. $\leq 0,00155$), todos $\leq 0,003$: el twirling no introduce sesgo. C1 se cumple en medio y alto (crudo 0,02958/0,03036/0,02997 frente a 0,030 y 0,04953/0,05046/0,04967 frente a 0,050, $\leq$

0,0005 de diferencia): el detector ve lo inyectado. Los valores de C1-C5 del manifiesto son los que registró el ejecutor; el juez los volverá a exigir.

ZNE y PEC (control C4 y celdas zne/pec, nivel medio). El estadístico de estas celdas es el agregado $|\langle Z \rangle|/2$ (no la media por qubit; ver aplicacion/ejecutor_e2.py), sin IC (intervalo degenerado).

técnica	semilla	crudo	tras la técnica	cambio relativo
ZNE	20261007	0,02958	0,03029	+2,4 %
ZNE	20261008	0,03036	0,03054	+0,6 %
ZNE	20261009	0,02997	0,03194	+6,6 %
PEC	20261007	0,02958	0,02956	-0,1 %
PEC	20261008	0,03036	0,03077	+1,3 %
PEC	20261009	0,02997	0,02971	-0,9 %

Ninguna técnica mueve el sesgo de lectura: el cambio relativo es $\leq 6,6\%$ ($<10\%$, el umbral de C4 para «sin efecto»), y en ZNE incluso apunta hacia más sesgo. Es el resultado que la preinscripción anticipó (§5.4) y que ya había mostrado S.02; **se reporta como SIN EFECTO ESPERADO, no como fallo.** (!) sin verificar que PEC corriera con perfil de puerta no trivial en esta corrida: aquí, con sólo ruido de lectura, su perfil es trivial.

mtree (contraste C5, no decide). Reduce el sesgo crudo a 0,00047-0,00192 en los niveles sintéticos (bajo: 0,00067/0,00047/0,00056; medio: 0,00081/0,00066/0,00081; alto: 0,00084/0,00192/0,00102) y a 0,00061-0,00123 en el realista. Es una cifra de cuasi-probabilidades, no de bits, y no es comparable término a término con la del twirling (que sí conserva bitstrings); que en el nivel alto mthree (0,00192, semilla 20261008) quede por encima del twirling (0,00103) no es un hallazgo preinscrito.

Desenlace de E2. PENDIENTE: veredicto de C.E2 (qrecauda juzgar E2). Con la lectura anterior, el resultado esperable es CUMPLE en los tres niveles sintéticos, con las salvedades de los puntos 2 y 3, pero esa es una expectativa, no un veredicto. Un punto que el juez podría resolver de otro modo: si trata C4 = verdadero (sin efecto) como lo hace el manifiesto.

7.3 C.E1a-d — calidad de la clave y controles de E1

PENDIENTE: corrida C.E1a (control negativo: PRNG clásico). **PENDIENTE: corrida C.E1b** (Aer ruidoso sin mitigar). **PENDIENTE: corrida C.E1c** (Aer ruidoso con twirling). **PENDIENTE: corrida C.E1d** (control positivo: fuentes sintéticas, instrumento). **PENDIENTE: veredicto de E1.**

No se reporta ninguna cifra de la clave (longitud, M1-M5 en cruda, mitigada y clave, 90B, proporción NIST) ni se estiman. Sólo existen las cifras de **calibración** de P1 (§6.4, spikes/S04_90b/calibracion_pl.json), que no son un resultado de E1. El único dato disponible de ejecutor es la existencia de pruebas de unidad e integración (tests/aplicacion/test_ejecutor_e1.py, tests/integracion/test_e1_real.py), que comprueban el ejecutor, no el pipeline con las tres semillas declaradas. Los marcadores se sustituirán por la tabla de la corrida cuando exista registro/corridas/E1a...E1d.json y su veredicto.

7.4 C.E3 — tasa, latencia y ciclo de cifrado

PENDIENTE: corrida C.E3 (perfil A, 3 semillas $\times$ (3 + 30) repeticiones; perfil B; controles U1-U5 y T4). **PENDIENTE: veredicto de E3.**

No se reporta ninguna cifra de M6, M7, su desglose por etapa ni del coeficiente de variación. La expectativa de §6.3 sobre M7 es una **hipótesis** y se decidirá por la corrida: si M7 no se

cumple, el veredicto dirá cuál métrica, por cuánto (cociente a umbral) y en qué etapa se va el tiempo; y el resultado del perfil B se reportará como «cumple a medias», sin aprobar E3.

7.5 Resumen de estado de las evidencias

bloque	corrida / spike	estado a 2026-10-07
Versiones del entorno	S.01	hecho
Alcance de la mitigación	S.02	hecho
Aer es pseudoaleatorio	S.03	hecho
Estimador SP 800-90B	S.04 (+ calibración P1)	hecho
E1 (C.E1a-d)	—	PENDIENTE
E2 (C.E2)	registro/corridas/C.E2*.json	corrida registrada, controles en verde; veredicto PENDIENTE
E3 (C.E3)	—	PENDIENTE
Hardware IBM real (F3.04, F3.06)	—	PENDIENTE (opcional)
TRL por componente (T. TRL)	—	PENDIENTE
Revisión adversarial del release (R.01)	—	PENDIENTE

8. Discusión

Lo que muestra la evidencia disponible. (i) El simulador es pseudoaleatorio (S.03): cualquier afirmación de «entropía cuántica» a partir de Aer sería falsa, y el diseño lo impide por tipo (el Origen). (ii) ZNE y PEC no actúan sobre el error de lectura (S.02 y las celdas de C.E2): la mitigación del bitstream de un QRNG exige una técnica propia que conserve los bits por disparo; el twirling con XOR clásico lo hace y, en las condiciones medidas, lleva el sesgo medio hasta la altura del ruido de muestreo. (iii) El estimador MCV es ciego a la dependencia y el mínimo de SP 800-90B es conservador: por eso la longitud de la clave y los criterios de control no pueden depender de un único estimador ni de un umbral absoluto (S.04, calibración de P1).

Por qué «certificada» es una palabra peligrosa. SP 800-22 es una batería estadística; un PRNG la pasa. Por eso C.E1a, el control negativo, está en el diseño como criterio de éxito y no como adorno: si el PRNG pasa M1-M5, el informe lo dirá, y un lector no podrá inferir de «la clave pasa NIST» que la fuente sea cuántica. Con la misma lógica, el rótulo «certificada» queda definido en el glosario como «supera la batería y las cotas de entropía de este repositorio», nunca como certificación formal.

El riesgo de medir sólo la salida. Toeplitz limpia lo que el ruido ensució: medir M1-M5 sólo sobre la clave hace que cualquier fuente pase (R.00-1). De ahí las tres posiciones de medida y el criterio B1 de C.E1b —la cruda debe fallar con el ruido declarado— y la lectura honesta del hallazgo «la clave de C.E1b pasa aunque la cruda falle», que se escribirá sea cual sea el desenlace.

La mitigación puede ser contraproducente en pruebas locales. Las enmiendas de M4/M5 en la mitigada reconocen que el twirling por bloques simetriza en media pero deja estructura local que las pruebas de bloques detectan. Es la parte menos satisfactoria del diseño: se decidió **antes de correr** y por un diagnóstico de escritorio, y marcar como informativa una prueba que podría fallar es una decisión que se puede cuestionar. Se mitiga porque la clave —lo que se cifra— sigue juzgándose con M1-M5 completas, y porque la corrida C.E1c registrará los

valores reales de M4 y M5 en la mitigada, de modo que la hipótesis del equipo se contrastará, no se supondrá. Si las pruebas de bloques fallaran de forma grave en la mitigada, el equipo deberá examinar si el sesgo local degrada también a Peres (que exige independencia).

Lectura de C.E2 sin triunfalismo. El criterio K1-K4 se diseñó para que una celda que falle no se absuelva con otras semillas. En la lectura del autor no falla ninguna. Pero tres hechos matizan: el nivel bajo es un empate con el umbral por construcción de P.E0; el nivel realista casi no tiene asimetría de lectura que corregir; y el piso del estadístico ($\approx 0,0006$) es del mismo orden que el residuo, de modo que $K3 = 0,003$ deja un margen de $\approx 5\times$ que habla más de la precisión del instrumento que de la potencia de la técnica. La conclusión sólida es la que la preinscripción afirma: en las condiciones del modelo sintético, el twirling baja el sesgo de lectura por debajo de M1 con holgura; no es una afirmación sobre hardware real.

Latencia: el resultado negativo probable. El diseño anticipa que M7 en el perfil A no se cumpla: validar con NIST y 90B sobre $\geq 10^6$ bits y un muestreo con un bucle Python por disparo no cabe en 500 ms. Se dejó escrito como hipótesis (!) y con un perfil B de contraste para no esconderlo. Si se confirma, el resultado útil es dónde se va el tiempo (por etapa) y qué debería cambiar (lote por adelantado, validación fuera de la ruta crítica); eso sería una nueva eureka con preinscripción propia.

9. Limitaciones

1. **El simulador no aporta entropía cuántica.** Sólo el hardware puede, y aun así su origen cuántico **no se certifica** sin pruebas de Bell o autoverificación, fuera de alcance. «Certificada» significa «supera esta batería».
2. **TRL real = el de la afirmación más débil.** La fuente cuántica en simulador es TRL 3; la rúbrica de TRL 4 exige repetibilidad $n \geq 3$, protocolo preinscrito y auditable por un tercero. El simulador sostiene el pipeline de postprocesamiento y cifrado; la fuente sólo sube con F3.06 (corrida real con job_id). El rótulo definitivo lo derivará el nodo T.TRL de la evidencia (pendiente); este informe no emite ninguno.
3. **Independencia entre bits y entre bloques.** Peres es exacto para bits IID; el LHL usa $n \cdot h_{\min}$ por bit como min-entropía del bloque (una heurística estándar, pero que con dependencia no se sigue de una estimación por muestra); la semilla de Toeplitz sale del mismo pool (D-004). Todo ello es hipótesis del modelo de ruido (!) sin verificar), no hecho medido. El cross-talk no está modelado.
4. $\varepsilon = 2^{-64}$ **es un parámetro de diseño, no una garantía.** SP 800-90B da una cota empírica, no una prueba; ninguna de las baterías es una certificación FIPS/ISO.
5. **El piso de P1 se fijó conociendo las semillas** (§6.4) y $n=11$ no es un intervalo de tolerancia; el 90B de los bits de Aer ruidoso/mitigado puede comportarse distinto del de PRNG (ese 90B sigue siendo informativo).
6. **Algunas pruebas de la mitigada son sólo informativas** por decisión previa a la corrida, basada en un diagnóstico de escritorio sin corrida registrada.
7. **El nivel realista no pone a prueba la mitigación** (casi no hay asimetría de lectura en el backend falso), y el nivel bajo es un empate con el umbral.
8. **Sólo un nivel de ruido en E1** (el medio), elección de la preinscripción, no de P.E0.
9. **Latencia: sólo en PC con Aer**, un hilo, sin cola ni red de IBM Quantum; perf_counter en WSL2 no coincide con el de Windows, de modo que las cifras valen para esta máquina y este reloj. No hay requisito externo que ancle 500 ms ni 10 kbit/s.
10. **Amenazas fuera de alcance:** el canal, canales laterales, gestión de claves, un atacante con acceso al pool.
11. **PNA y Samplomatic no están implementados** (exigen ruido aprendido en el servicio de IBM); PEC corre con perfil de puerta conocido, no aprendido.
12. **Dependencias frágiles:** el binario 90B se compila fuera del repo (red y g++, -march=native: no portable entre CPUs); mthree y nistrng se fijan por versión exacta.

13. **El diseño se ha revisado una vez** (R.00, adversarial, sobre el diseño); la revisión del release (R.01) sigue pendiente.
14. **Las cifras de C.E2 no tienen veredicto** (§7.2); y el juez, no este informe, decide.

10. Reproducibilidad

10.1 Entorno y versiones

Python 3.13.14 (requires-python >= 3.11). Versiones del entorno en el que se corrió C.E2 (manifiesto) y S.01: qiskit 2.5.2, qiskit-aer 0.17.2, qiskit-ibm-runtime 0.50.0, mthree 3.0.0, nistrng 1.2.3, cryptography 50.0.2, numpy 2.5.3, scipy 1.18.1. Las versiones exactas están en **uv.lock** (168 paquetes; SHA-256 del fichero a este borrador: f3694ee177c6a44c4e4eec762a8bbe9df5cb3a4afe último commit que lo toca: a5bf535) y en requirements.txt. SP 800-90B: etiqueta v1.1.8 de usnistgov/SP800-90B_EntropyAssessment.

10.2 Comandos exactos

```
git clone https://github.com/jramirezgen/qrecauda.git && cd qrecauda
uv sync --group dev --extra cuantico --extra mitigacion --extra validacion --extra cifrado
bash spikes/S04_90b/build_nist.sh # compila ea_non_iid en /tmp/qrecauda_nist90b (r
./scripts/ci_local.sh # ruff · mypy · lint-imports · DAG · pytest
```

Bala trazadora con el PRNG de línea base

```
uv run --no-sync qrecauda
```

Spikes

```
uv run --no-sync python spikes/S01_versiones/run.py
uv run --no-sync python spikes/S02_mitigacion_alcance/run.py
uv run --no-sync python spikes/S03_aer_pseudoaleatorio/run.py
uv run --no-sync python spikes/S04_90b/run.py
uv run --no-sync python spikes/S04_90b/calibracion_pl.py
```

Eureka: desde la declaración al veredicto (la preinscripción debe preceder a la corrida)

```
uv run --no-sync qrecauda correr declaraciones/E1.toml && uv run --no-sync qrecauda juzgar E1
uv run --no-sync qrecauda correr declaraciones/E2.toml && uv run --no-sync qrecauda juzgar E2
uv run --no-sync qrecauda correr declaraciones/E3.toml && uv run --no-sync qrecauda juzgar E3
```

uv run sin --no-sync puede quitar los extras; por eso se usa --no-sync (o .venv/bin/python) tras sincronizar. Las corridas pesadas toman el **candado de máquina** (salidas/candado_maquina.lock), fuerzan **BLAS a un hilo** y registran el entorno en el manifiesto; E3 mide tiempo y exige una máquina libre. Dentro de una corrida, cada celda es determinista dada su semilla (el muestreo de Aer es un PRNG). Los comandos de E1 y E3 se listan para completar la cadena; **no hay todavía resultados registrados de ellos** (§7.3, §7.4).

10.3 Commits relevantes (el historial es el testigo del orden)

qué	commit
Dominio: extractores, entropía, métricas	0419140
Spikes S.01-S.04	7856f4e
P.E0 parámetros	f8e21d0
Validador NIST y estimador 90B	42b54c9
Cifrador AES-GCM y reserva	aea23de
Mitigación twirling (F4.01)	a9bd306

qué	commit
ZNE y PEC sobre ⟨Z⟩ (F4.02)	ce52c0d
Preinscripción de E2	43e2649
Preinscripción de E3	d0f062a
Preinscripción de E1	dae22a6
Guard de cifrado = M1-M5 (F6.02)	8984147
Ejecutor real de E2 y E3	26e5b41
Enmiendas de E1 (M4; P1) y juez de E1	1a06602, 37b89ca, ce8ba23, 447c907
Prueba de integración Aer → ... → AES-GCM	013fd62
Licencia Apache-2.0	fb20457

Orden verificable: el commit de la preinscripción de E2 (43e2649) precede al del ejecutor (26e5b41) y la corrida registra ambos (preinscripcion_sha 43e26492..., commit 26e5b41d...). El repositorio tiene 48 commits a la fecha de este borrador. Las enmiendas de E1 y E3 son anteriores a cualquier corrida de esas eurekas (no existe ninguna en el registro).

10.4 Lo que sí y lo que no es reproducible

Sí: el pipeline completo con semillas fijas (determinista), las tablas de §7 (desde registro/corridas/ y los resultado.json de los spikes), y el orden de preinscripción. No: **la entropía no es reproducible por definición** —en simulación, lo reproducible es justamente lo opuesto a lo que un QRNG real debe dar—; los tiempos de E3 son de una máquina y un reloj; y el binario 90B depende de la CPU.

11. Conclusiones

1. Se construyó un pipeline QRNG de punta a punta (fuente → mitigación → Peres → Toeplitz/LHL → validación → AES-256-GCM) con límites declarados: el simulador valida el postprocesamiento y **no** aporta entropía cuántica; «certificada» significa «supera esta batería».
2. Los spikes establecieron hechos que cambian el diseño: Aer es pseudoaleatorio (S.03); ZNE y PEC no actúan sobre el error de lectura ni producen bitstrings, y mthree entrega cuasi-probabilidades (S.02); el MCV es ciego a la dependencia y el mínimo del 90B es conservador (S.04).
3. La mitigación propia por twirling con XOR clásico conserva los bits por disparo; en la lectura del autor de la corrida C.E2 (400 000 disparos × 8 qubits, tres semillas), baja el sesgo medio por qubit de 0,0098–0,0505 a 0,00074–0,00131 en los tres niveles sintéticos, mientras que ZNE y PEC no lo mueven ($\leq 6,6\%$). **El veredicto de E2 está pendiente.**
4. **E1 (calidad de la clave, con control negativo con PRNG y control positivo de fuentes defectuosas) y E3 (tasa y latencia, con el perfil B de contraste) están pendientes;** no se afirma nada de sus resultados. Se anticipa, como hipótesis, que M7 no se cumpla en el perfil que decide.
5. El método —preinscripción con commit anterior a la corrida, controles sin los cuales no hay veredicto, enmiendas fechadas y la confesión de que el piso de P1 se calibró conociendo las semillas— es la contribución que sobrevive a cualquier resultado, positivo o negativo.
6. El TRL real es el de la afirmación más débil (fuente en simulador: TRL 3). Para subir hace falta una corrida en hardware IBM real con job_id y, aun entonces, el origen cuántico no se certificaría sin pruebas de Bell.

Trabajo siguiente (en este orden): emitir el veredicto de E2; correr C.E1a–d y C.E3 y sustituir los marcadores de §7.3–7.4; derivar T.TRL; revisión adversarial del release (R.01); y, si hay acceso, corrida en hardware (F3.06) con la mitigación de SamplerV2 de IBM comparada con el twirling propio.

Referencias

Las referencias se contrastaron con el registro de DOI de Crossref (o, el preprint de Qiskit, con la API de arXiv) el 2026-10-07. Para NIST se da el DOI del informe.

- [1] M. Herrero-Collantes y J. C. García-Escartín, «Quantum random number generators», *Reviews of Modern Physics* **89**, 015004 (2017). DOI 10.1103/RevModPhys.89.015004.
- [2] L. E. Bassham III, A. L. Rukhin, J. Soto, J. R. Nechvatal et al., «A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications», NIST Special Publication 800-22 Rev. 1a (2010). DOI 10.6028/NIST.SP.800-22r1a.
- [3] M. S. Turan, E. Barker, J. Kelsey, K. A. McKay et al., «Recommendation for the Entropy Sources Used for Random Bit Generation», NIST Special Publication 800-90B (2018). DOI 10.6028/NIST.SP.800-90B.
- [4] Y. Peres, «Iterating von Neumann’s procedure for extracting random bits», *The Annals of Statistics* **20**(1), 590–597 (1992). DOI 10.1214/aos/1176348543.
- [5] R. Impagliazzo, L. A. Levin y M. Luby, «Pseudo-random generation from one-way functions», *Proc. 21st ACM STOC*, 12–24 (1989). DOI 10.1145/73007.73009.
- [6] J. Håstad, R. Impagliazzo, L. A. Levin y M. Luby, «A pseudorandom generator from any one-way function», *SIAM Journal on Computing* **28**(4), 1364–1396 (1999). DOI 10.1137/S0097539793244708.
- [7] H. Krawczyk, «LFSR-based hashing and authentication», en *Advances in Cryptology — CRYPTO ’94*, LNCS, 129–139. DOI 10.1007/3-540-48658-5_15.
- [8] S. Pironio, A. Acín, S. Massar, A. Boyer de la Giroday et al., «Random numbers certified by Bell’s theorem», *Nature* **464**, 1021–1024 (2010). DOI 10.1038/nature09008.
- [9] D. A. McGrew y J. Viega, «The Security and Performance of the Galois/Counter Mode (GCM) of Operation», en *Progress in Cryptology — INDOCRYPT 2004*, LNCS, 343–355 (2004). DOI 10.1007/978-3-540-30556-9_27.
- [10] M. Dworkin, «Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC», NIST Special Publication 800-38D (2007). DOI 10.6028/NIST.SP.800-38D.
- [11] A. Javadi-Abhari et al., «Quantum computing with Qiskit», arXiv:2405.08810 (2024). El simulador Aer (qiskit-aer) se cita por su software y versión (0.17.2); no se cita un artículo propio.
- [12] K. Temme, S. Bravyi y J. M. Gambetta, «Error mitigation for short-depth quantum circuits», *Physical Review Letters* **119**, 180509 (2017). DOI 10.1103/PhysRevLett.119.180509.
- [13] Y. Li y S. C. Benjamin, «Efficient variational quantum simulator incorporating active error minimization», *Physical Review X* **7**, 021050 (2017). DOI 10.1103/PhysRevX.7.021050.
- [14] E. van den Berg, Z. K. Mineev y K. Temme, «Model-free readout-error mitigation for quantum expectation values», *Physical Review A* **105**, 032620 (2022). DOI 10.1103/PhysRevA.105.032620.
- [15] P. D. Nation, H. Kang, N. Sundaresan y J. M. Gambetta, «Scalable mitigation of measurement errors on quantum computers», *PRX Quantum* **2**, 040326 (2021). DOI 10.1103/PRXQuantum.2.040326.

Nota sobre referencias no incluidas. El procedimiento original de von Neumann (1951) y los resultados clásicos sobre la universalidad de las matrices de Toeplitz (distintos de [7]) no se citan porque no pude contrastar sus datos bibliográficos en esta sesión. La caracterización de Qiskit Aer como PRNG se apoya en el spike S.03 y no en una referencia externa.

Apéndice A. Mapa de fuentes del repositorio

tema	fuentes en el repositorio
Objetivo, alcance, discrepancias	docs/FUNDAMENTO.md
Arquitectura	docs/DISEÑO.md
Amenazas	docs/AMENAZAS.md
Decisiones D-001...D-009	docs/decisiones/
Vocabulario	docs/GLOSARIO.md
Preinscripciones P.E0-P.E3	docs/preinscripciones/ y declaraciones/*.toml
Spikes	spikes/S01_versiones, S02_mitigacion_alcance, S03_aer_pseudoaleatorio, S04_90b
Revisión adversarial R.00	docs/informes/REVISION_DISEÑO_R00.md
Corridas	registro/corridas/
Estado generado	ESTADO.md, registro/nodos.jsonl

Apéndice B. Decisiones de diseño (resumen)

id	decisión
D-001	El backend es un puerto: simulador por defecto, hardware opcional
D-002	El simulador no reclama origen cuántico; sólo <code>HARDWARE_IBM</code> lo reclama
D-003	La mitigación se aplica donde su definición tiene sentido (lectura sobre bitstrings, ZNE/PEC sobre $\langle Z \rangle$)
D-004	Peres + Toeplitz con la semilla del mismo pool ($\epsilon = 2^{-64}$)
D-005	M1-M7 en un solo sitio, desigualdad estricta; «certificada» = supera esta batería
D-006	Dos validadores independientes que se contrastan
D-007	Cada eureka lleva un control negativo
D-008	Secretos por ruta, nunca por valor
D-009	F4.01 es twirling de lectura propio; <code>mthree</code> sólo de contraste; ZNE y PEC fuera del bitstream